

SESIÓN N° 02:

Módulos

I OBJETIVOS

- ❖ Conocer los principios básicos que rigen el uso de módulos en Python.
- ❖ Aprender a diseñar módulos en Python.
- ❖ Aprender las características y ventajas provistas por los módulos en la solución de problemas.

II TEMAS A TRATAR

- ❖ Introducción.
- ❖ ¿Qué es un módulo?
- ❖ ¿Cómo importar un módulo?
- ❖ Funcionamiento de la importación
- ❖ Módulos Incorporados
- ❖ Resumen

III MARCO TEORICO

1. INTRODUCCIÓN

Como observamos en capítulos anteriores, uno no puede desarrollar una aplicación compleja sin organizar nuestro código en funciones, y mientras que nuestro programa continúa creciendo cada vez más y más, vamos a necesitar una manera para organizarlo a un nivel que no se vuelva engorroso de mantener, es ahí donde viene a la mano los módulos. Con ellos podemos organizar un grupo de funciones como si fueran un conjunto de herramientas las cuales podemos hacer uso de ellas cada vez que las necesitemos.

Los módulos son bastante fáciles de crear, ya que son scripts sencillos de Python. Simplemente debemos de crearlos y luego invocarlos tal como se hacía con las funciones en los puntos y programas donde sean necesarios y requeridos.

El diseño modular de código es una estrategia que tiene muchas ventajas, comprender en cada lenguaje de programación como afecta al código es algo muy importante, al saber porque estamos organizando los programas o aplicaciones de forma modular asegura la forma como es que los programas se ejecutan haciendo uso de los recursos del lenguaje que se esté empleando. Los módulos son cargados en tres fases por el Lenguaje Python, las tres fases, este tipo de proceso nos lleva a usar un path o ruta de búsqueda, esto se lleva a cabo para poder posible la ubicación del recurso en su proceso de ejecución. Posteriormente se debe hacer la generación del bytecode asociado al mismo módulo que se está cargando. El último paso durante la importación es la ejecución del código del módulo importado.

2. ¿QUÉ ES UN MÓDULO?

Los módulos en Python son grupos de funciones alojadas dentro de un archivo .py.

Vamos a necesitar abrir un nuevo archivo en un editor de texto o en un IDE de tu preferencia. Para que el curso tenga sentido, voy a continuar con el ejemplo que utilizamos en el último capítulo, crear un carrito de compra (ecommerce). Solo que esta vez vamos a calcular el impuesto de un artículo en específico dado su precio.

```
def calcularImpuesto(precio, impuesto):  
    precioNuevo = precio / 100 * (100 + impuesto)    return precioNuevo
```

Observemos que fue sencillo de realizar, lo que hicimos fue definir una función llamada calcularImpuesto la cual toma dos argumentos precio e impuesto, para luego calcular y retornar el valor total del artículo.

Para que este código sea o cumpla como un módulo en Python, lo único que tenemos que hacer es guardarlo con la extensión .py en el mismo directorio donde residen nuestros otros scripts de la aplicación. Vamos a guardar el archivo con el siguiente nombre: finanzas.py y así nuestro módulo será el módulo de finanzas.

Para usar los módulos existen dos posibilidades, podemos usar la palabra clave import o la palabra clave from. import es la manera más simple y más común de usar, ya que importa todas las funcionalidades existentes dentro del archivo .py en donde este alojado ese módulo. Veamos como importar el ejemplo anterior en donde teníamos el módulo de finanzas (finanzas.py):

```
import finanzas
```

Observemos que para importar el módulo no hizo falta escribir la extensión del archivo. Una vez que Python interprete esta línea de código podemos tener a nuestra disposición la función calcularImpuesto que definimos anteriormente.

La otra manera es usando la palabra clave from, la cual importará única y exclusivamente la función que uno le pasa como parámetro, es decir, imaginemos que tenemos un módulo que posee miles y miles de funciones, para que importar todas esas funciones dentro de nuestro script si solo vamos a necesitar una, es en este caso donde viene a la mano el from. Una de las prácticas más recomendadas dentro de Python es hacer uso de la palabra from cuando se está trabajando con módulos, ya que salva tiempo de procesamiento y recursos de la máquina en cuestión.

Veamos como importar una funcionalidad del módulo de finanzas:

```
from finanzas import calcularImpuesto
```

Observemos que necesitamos especificar de qué módulo queremos obtener nuestra función con from + MÓDULO y luego la función que queremos importar import + FUNCIÓN. En nuestro caso, del módulo de finanzas importa la función calcularImpuesto.

También puedes importar varias funciones a la vez, solo necesitas separarlas por comas.

```
from finanzas import calcularImpuesto, calcularDescuento
```

Incluso puedes importar todas las funciones usando un wildcard

```
from finanzas import *
```

Después de importar los módulos, la manera de usar las funciones es la siguiente: el nombre del módulo, seguido de un punto, seguido del nombre de la función a usar. Veamos un ejemplo:

```
import finanzas print finanzas.calcularImpuesto(100, 5)
```

Si corremos el script, debería imprimir 105, lo que significa que significa que nuestro módulo funcionó a la perfección.

3. ¿CÓMO IMPORTAR UN MÓDULO?

Un formulario es un elemento de presentación que actúa como contenedor de un conjunto de controles (botones, cajas de texto, casillas de verificación, botones radio, etc.), le permiten al usuario ingresar datos de una forma pre-establecida y enviarlos al servidor web para su procesamiento. Los formularios con HTML amplía no solo el formateo de etiquetas, sino que incluye objetos gráficos y controles.

4. FUNCIONAMIENTO DE LA IMPORTACIÓN

Algunos lenguajes permiten trabajar de forma estructurada de forma análoga a como lo hace Python. Se pueden crear componentes y desde unos ficheros invocar a funciones y/o variables declaradas en otros. Sin embargo, a diferencia, por ejemplo, del lenguaje C. en Python la importación de un módulo no es simplemente la inserción de código de un fichero en otro, ya que esta es una operación que ocurre en tiempo de ejecución. En concreto, se llevan a cabo tres pasos durante la importación. El primero de ellos localiza el fichero físico de código que corresponde al módulo en cuestión. Para llevar a cabo este paso, se utiliza un path de búsqueda determinado del que nos ocuparemos en el apartado siguiente de este capítulo. Una vez localizado el fichero se procede a la Generación del bytecode asociado al mismo. Este proceso ocurre en función de las fechas del fichero de código (.py) y del bytecode asociada (.pyc). Si la del primero es posterior a la del segundo, entonces se vuelve a generar todo automáticamente.

El último paso durante la importación es la ejecución del código del módulo importado. Esta es la razón por la cual se ejecuta la sentencia print de nuestro ejemplo. Dado que los pasos llevados a cabo durante la importación pueden ser costosos en tiempo, el intérprete de Python solo importa cada módulo una vez por proceso. Esto implica que sucesivas importaciones se saltarán los tres pasos y el Intérprete pasará directamente a utilizar el módulo contenido en memoria. No obstante, siendo este el comportamiento por defecto, Python nos permite emplear la función reload del módulo imp, que se encuentra en la librería estándar. Gracias a la cual, es posible indicar que se vuelva a realizar la importación de un determinado módulo.

5. MÓDULOS INCORPORADOS

Existen miles de módulos incorporados dentro de Python. Debido a que el rango es extenso, es imposible cubrirlos todos dentro de este curso, por lo que solo me voy basar en los más útiles.

Si quieres conocer más sobre los módulos incorporados en Python te invito a que le eches un vistazo a la página oficial donde encontrarás toda la documentación necesaria. Veamos cuales son esos módulos:

- `math`
- `os`
- `datetime` ▪ `urllib2`

A. MATH

El módulo de `math` nos provee el acceso a funciones y constantes matemáticas. Veamos unos ejemplos:

```
import math

print(math.pi) #Pi, 3.14...
print(math.e) #Número de Euler, 2.71...

print(math.degrees(2)) #2 radianes = 114.59 grados
print(math.radians(60)) #60 grados = 1.04 radianes
print(math.sin(2)) #Seno de 2 radianes
print(math.cos(0.5)) #Coseno de 0.5 radianes
print(math.tan(0.23)) #Tangente de 0.23 radianes
print(math.factorial(5)) #1 * 2 * 3 * 4 * 5 = 120
print(math.sqrt(49)) #Raíz cuadrada de 49 = 7
```

Existe miles de funciones dentro de este módulo te invito a que las revises todas aquí.

B. DATETIME

Si necesitas trabajar con fechas y tiempos, entonces el módulo `datetime` es el indicado para ti. Veamos un ejemplo con los usos más comunes:

```
import datetime
from datetime import date
import time

print(time.time()) #Retorna el número de segundos desde el 1 de enero de 1970 (Unix Epoch)
print(date.fromtimestamp(123456789)) #Convierte número de segundos en un objeto tipo date
print(date.fromtimestamp(time.time())) #Podemos combinar las funciones time y fromtimestamp para representar el tiempo actual
currentDate = date.fromtimestamp(time.time())
print(currentDate) #Creamos una variable con la representación del tiempo actual
print(currentDate.strftime("%d/%m/%y")) #Le coloca el siguiente formato a la fecha DD/MM/YY
print(currentDate.isoformat()) #Le coloca el formato estándar ISO a esa fecha
print(currentDate.isoformat()) #Le coloca el formato estándar ISO a esa fecha
```

Existe miles de funciones dentro de este módulo te invito a que las revises todas aquí.

C. OS

Este módulo te permite trabajar con el sistema operativo en el cual Python este ejecutándose, ya sea Windows, Mac o Linux. Nos enfocaremos en la funcionalidad `path` ya que es la más común. `Path` nos permite manipular y encontrar propiedades de los archivos y carpetas que existen en el sistema. Vemos unos ejemplos:

```
from os import path from os import path
print(path.exists("C:\\Users\\uno.txt"))
print(path.getatime("C:\\Users")) #Retorna la fecha en la cual ese directorio fue
accedido por ultima vez
print(path.getmtime("C:\\Users")) #Retorna la fecha en la cual ese directorio fue
modificado por ultima vez
print(path.getsize("C:\\Users\\angelMon\\Documents\\Guia4_GoogleEarth.pdf") )
#Retorna el tamaño en bytes de ese archivo

path.join("C:", "Users") #Retorna la siguiente dirección "C:/Users"
```

Existe miles de funciones dentro de este módulo te invito a que las revises todas aquí.

6. PAQUETES

Los paquetes son una manera de estructurar los espacios de nombres de Python usando "nombres de módulos con puntos". Por ejemplo, el nombre de módulo A.B designa un submódulo llamado B en un paquete llamado A. Tal como el uso de módulos evita que los autores de diferentes módulos tengan que preocuparse de los respectivos nombres de variables globales, el uso de nombres de módulos con puntos evita que los autores de paquetes de muchos módulos, como NumPy o la Biblioteca de Imágenes de Python (Python Imaging Library, o PIL), tengan que preocuparse de los respectivos nombres de módulos. Suponte que quieres designar una colección de módulos (un "paquete") para el manejo uniforme de archivos y datos de sonidos. Hay diferentes formatos de archivos de sonido (normalmente reconocidos por su extensión, por ejemplo: .wav, .aiff, .au), por lo que tienes que crear y mantener una colección siempre creciente de módulos para la conversión entre los distintos formatos de archivos. Hay muchas operaciones diferentes que quizás quieras ejecutar en los datos de sonido (como mezclarlos, añadir eco, aplicar una función ecualizadora, crear un efecto estéreo artificial), por lo que además estarás escribiendo una lista sin fin de módulos para realizar estas operaciones. Aquí hay una posible estructura para tu paquete (expresados en términos de un sistema jerárquico de archivos):

Al importar el paquete, Python busca a través de los directorios en sys.path, buscando el subdirectorio del paquete.

Los archivos init .py se necesitan para hacer que Python trate los directorios como que contienen paquetes; esto se hace para prevenir directorios con un nombre común, como string, de esconder sin intención a módulos ~~válidos~~ que se suceden luego en el camino de búsqueda de módulos. En el caso más simple, init .py puede ser solamente un archivo vacío, pero también puede ejecutar código de inicialización para el paquete o configurar la variable all , descrita luego.

```
sound/                                Top-level package
__init__.py                          Initialize the sound package
formats/                             Subpackage for file format conversions
    __init__.py
    wavread.py
    wavwrite.py
    aiffread.py
    aiffwrite.py
    auread.py
    auwrite.py
    ...
effects/                             Subpackage for sound effects
    __init__.py
    echo.py
    surround.py
    reverse.py
    ...
filters/                             Subpackage for filters
    __init__.py
    equalizer.py
    vocoder.py
    karaoke.py
    ...
```

Los usuarios del paquete pueden importar módulos individuales del mismo, por ejemplo:

```
import sound.effects.echo
```

Esto carga el submódulo `sound.effects.echo`. Debe hacerse referencia al mismo con el nombre completo.

```
sound.effects.echo.echofilter(input, output, delay=0.7, atten=4)
```

Otra alternativa para importar el submódulo es:

```
from sound.effects import echo
```

Esto también carga el submódulo `echo`, lo deja disponible sin su prefijo de paquete, por lo que puede usarse así:

```
echo.echofilter(input, output, delay=0.7, atten=4)
```

Otra variación más es importar la función o variable deseadas directamente:

```
from sound.effects.echo import echofilter
```

De nuevo, esto carga el submódulo `echo`, pero deja directamente disponible a la función `echofilter()`:

```
echofilter(input, output, delay=0.7, atten=4)
```

Note que al usar `from package import item` el ítem puede ser tanto un submódulo (o subpaquete) del paquete, o algún otro nombre definido en el paquete, como una función, clase, o variable. La declaración `import` primero verifica si el ítem está definido en el paquete; si no, asume que es un módulo y trata de cargarlo. Si no lo puede encontrar, se genera una excepción `ImportError`.

Por otro lado, cuando se usa la sintaxis como `import item.subitem.subsubitem`, cada ítem excepto el último debe ser un paquete; el mismo puede ser un módulo o un paquete pero no puede ser una clase, función o variable definida en el ítem previo.

A. IMPORTANDO * DESDE UN PAQUETE

Ahora, ¿qué sucede cuando el usuario escribe `from sound.effects import *`? Idealmente, uno esperaría que esto de alguna manera vaya al sistema de archivos, encuentre cuales submódulos están presentes en el paquete, y los importe a todos. Esto puede tardar mucho y el importar sub-módulos puede tener efectos secundarios no deseados que sólo debiesen ocurrir cuando se importe explícitamente el sub-módulo.

La única solución es que el autor del paquete provea un índice explícito del paquete. La declaración `import` usa la siguiente convención: si el código del `__init__.py` de un paquete define una lista llamada `__all__`, se toma como la lista de los nombres de módulos que deberían ser importados cuando se hace `from package import *`. Es tarea del autor del paquete mantener actualizada esta lista cuando se libera una nueva versión del paquete. Los autores de paquetes podrían decidir no soportarlo, si no ven un uso para importar `*` en sus paquetes. Por ejemplo, el archivo `sounds/effects/__init__.py` podría contener el siguiente código:

```
__all__ = ["echo", "surround", "reverse"]
```

Esto significaría que `from sound.effects import *` importaría esos tres submódulos del paquete `sound`.

Si no se define `__all__`, la declaración `from sound.effects import *` no importa todos los submódulos del paquete `sound.effects` al espacio de nombres actual; sólo se asegura que se haya importado el paquete `sound.effects` (posiblemente ejecutando algún código de inicialización que haya en `__init__.py`) y luego importa aquellos nombres que estén definidos en el paquete. Esto incluye cualquier nombre definido (y submódulos explícitamente cargados) por `__init__.py`. También incluye cualquier submódulo del paquete que pudiera haber sido explícitamente cargado por declaraciones `import` previas. Considerá este código:

```
import sound.effects.echo
import sound.effects.surround
from sound.effects import *
```

En este ejemplo, los módulos `echo` y `surround` se importan en el espacio de nombre actual porque están definidos en el paquete `sound.effects` cuando se ejecuta la declaración `from...import`. (Esto también funciona cuando se define `__all__`).

A pesar de que ciertos módulos están diseñados para exportar solo nombres que siguen ciertos patrones cuando usás `import *`, también se considera una mala práctica en código de producción.

Recuerda que no está mal usar `from paquete import submodulo_especifico`. De hecho, esta notación se recomienda a menos que el módulo que estás importando necesite usar submódulos con el mismo nombre desde otros paquetes.

B. REFERENCIAS INTERNAS EN PAQUETES

Los submódulos frecuentemente necesitan referirse unos a otros. Por ejemplo, el módulo `surround` quizás necesite usar el módulo `echo module`. De hecho, tales referencias son tan comunes que la declaración `import` primero mira en el paquete actual antes de mirar en el camino estándar de búsqueda de módulos. Por lo tanto, el módulo `surround` puede simplemente hacer `import echo` o `from echo import echofilter`. Si el módulo importado no se encuentra en el paquete actual (el paquete del cual el módulo actual es un submódulo), la declaración `import` busca en el nivel superior por un módulo con el nombre dado.

Cuando se estructuran los paquetes en subpaquetes (como en el ejemplo `sound`), podés usar `import` absolutos para referirte a submódulos de paquetes hermanos. Por ejemplo, si el módulo `sound.filters.vocoder` necesita usar el módulo `echo` en el paquete `sound.effects`, puede hacer `from sound.effects import echo`.

Desde Python 2.5, además de los `import` relativos implícitos descritos arriba, podés escribir `import` relativos explícitos con la declaración de la forma `from module import name`. Estos `import` relativos explícitos usan puntos adelante para indicar los paquetes actual o padres involucrados en el `import` relativo. En el ejemplo `surround`, podrías hacer:

```
from . import echo
from .. import formats
from ..filters import equalizer
```

Note que ambos import, relativos explícitos e implícitos, se basan en el nombre del módulo actual. Ya que el nombre del módulo principal es siempre "`__main__`", los módulos pensados para usarse como módulo principal de una aplicación Python siempre deberían usar import absolutos.

C. PAQUETES EN MÚLTIPLES DIRECTORIOS

Los paquetes soportan un atributo especial más, `path_`. Este se inicializa, antes de que el código en ese archivo se ejecute, a una lista que contiene el nombre del directorio donde está el paquete. Esta variable puede modificarse, afectando búsquedas futuras de módulos y subpaquetes contenidos en el paquete.

Aunque esta característica no se necesita frecuentemente, puede usarse para extender el conjunto de módulos que se encuentran en el paquete.

7. RESUMEN

En esta lección, aprendimos como crear módulos en Python, como importarlos en nuestros programas, y como usar los módulos que vienen incorporados en el lenguaje. Te invito a que practiques y crees tus propios módulos ya que lo mejor de todo es que los puedes reutilizar en diferentes aplicaciones que estés desarrollando. Si tienes alguna pregunta, yo estaré feliz de responderte en los comentarios, además espero que te unas y le eches un vistazo a todo el resto de este curso.

Las normas de carga de módulos se hacen en tres fases en lenguaje Python, se inicia con la utilización de una ruta de búsqueda, la cual nos permitirá ubicar a los módulos a emplear, la segunda fase corresponde a la generación de un bytecode, las normas de generación dependen de la coherencia entre las fechas de compilación entre este y otros recursos y finalmente la ejecución del módulo.

La organización de un proyecto en módulos nos provee mayor independencia funcional entre funciones y módulos al utilizarlos no solo organizamos mejor nuestro proyecto, sino que también es posible optimizar su uso y la reutilización de código de una forma muy simple y práctica, reduciendo así la cantidad de código y esfuerzo, como también reducimos las complicaciones en su funcionamiento. Otro punto importante es el de desarrollo en equipo, los paquetes evitan la colisión de nombres, es decir que un conjunto de módulos puede usar sus propios identificadores para sus recursos sin preocuparse por los nombres de otro módulo, la organización por paquetes evita las confusiones de ámbito y dominio.

(La práctica tiene una duración de 2 horas) ACTIVIDADES**1. EXPERIENCIA DE PRÁCTICA N° 01: CREAR MÓDULOS**

Conforme un grupo tal como lo indica el docente.

Crear una cuenta de GitHub donde creará y registrará todos sus proyectos:

a) CREAR Y PROBAR UN MÓDULO:

1. Abra el editor de código del Python 3.7(IDE):
2. crear funciones para hallar el máximo de 2 valores, el mínimo de 2 valores, comprobar si los valores son iguales y comprobar si los valores son diferentes.
3. grabar el archivo de código bajo el nombre de funcionesComparacion.py.
4. Elaborar un programa aparte bajo el nombre de pruebaFunciones.py, importar el módulo creado en la pregunta número 2, probar las funciones presentes en dicho módulo.

b) PROBAR DIFERENTES FORMAS DE IMPORTACIÓN DE MÓDULOS:

1. Crear otro archivo de código bajo el nombre de funcionesIngreso.py.
2. Escribir una función que nos permita ingresar un valor y verificar si es un número, mostrando un mensaje de error cuando el valor no es un número al no tener valores entre 0 y 9 para cada uno de sus dígitos.
3. Grabar el módulo adecuadamente.
4. En el archivo prueba funciones importar únicamente la función de ingreso y no el módulo completo.
5. Ejecutar y probar la aplicación.

c) CREACIÓN DE UN MÓDULO DE MENÚ:

1. Crear un nuevo archivo y grabarlo con el nombre de menú.py.
2. Diseñar un menú de opciones.
3. Crear una función adicional que nos permita llamar al menú de opciones.
4. Crear una función adicional cuya función es llamar a la función elegida dentro del menú de opciones.
5. Grabar y probar adecuadamente el código desarrollado.

2. EXPERIENCIA DE PRÁCTICA N° 02: USANDO PATHS**a) ESTABLECER UNA CARPETA DE PROYECTO**

1. Crear un directorio en donde se almacenen todos los archivos de código del proyecto que se está realizando.
2. Determinar formalmente cuál es la ruta de la carpeta para incluirla en los proyectos sucesivos.
3. Cambiar los códigos de importación de módulos utilizando las rutas absolutas de los archivos establecidos en el proyecto.
4. Compilar y ejecutar la aplicación.

3. EXPERIENCIA DE PRÁCTICA N° 03: GESTIÓN DE PAQUETES**a) CREAR PAQUETES**

1. Los paquetes pueden contener módulos y otros paquetes. Son directorios. El único requisito es que contengan un archivo llamado `__init__.py`. Este archivo puede estar vacío.
2. La sentencia `from` se utiliza en conjunto a la previa sentencia `import` para importar un módulo.

```
>>> from utilidades import suma_total
```

Laboratorio de Lenguajes de Programación 1

3. Por ejemplo, cree un directorio llamado `tostadas_pipo`, que contiene los archivos llamados `__init__.py`, `principal.py` (dentro del mismo directorio).
4. Archivo `__init__.py`, este archivo no tiene ningún contenido.
5. Archivo `principal.py` incluye el siguiente código:

```
from utilidades import impuestos
from utilidades import calculos

monto = int(input("Introduzca un monto entero: "))
# Llama función definida en el módulo "impuestos"
print "El impuesto IVA de 12%:", impuestos.impuesto_iva12(monto)

suma = int(input("Introduzca un monto entero a sumar: "))
# Llama función definida en el módulo "calculos"
print "La suma total es:", calculos.suma_total(suma)
```

6. Seguidamente dentro del directorio `tostadas_pipo`, cree otro directorio llamado `utilidades`, dentro de este, cree los siguientes archivos:
7. Archivo `__init__.py`, este archivo no tiene ningún contenido.
8. Archivo `calculos.py` incluye el siguiente código:

```
def suma_total(monto=0):
    """ Calcula la suma total """
    calculo_suma = 20
    calculo_suma += monto
```

Archivo `impuestos.py` incluye el siguiente código:

9. Al final tendrá la siguiente estructura del directorio del paquete Python llamado `tostadas_pipo`, como se describe a continuación:

```
def impuesto_iva12(monto=0):
    """ Calcula el impuesto del IVA de 12 % """
    total = ((monto * 12)/100)
    return total

def impuesto_iva14(monto=0):
    """ Calcula el impuesto del IVA de 14 % """
    total = ((monto * 14)/100)
```

```
tostadas_pipo/
├── __init__.py
├── principal.py
├── utilidades/
│   ├── calculos.py
│   ├── impuestos.py
│   └── __init__.py
```

10. Entonces realizar importaciones desde una estructura de directorios mas completa se realiza de las siguientes formas:
11. Importar todos los módulos el sub-paquete `utilidades`, ejecutando:

```
import tostadas_pipo.utilidades
from tostadas_pipo import utilidades
from tostadas_pipo.utilidades import *
```

Laboratorio de Lenguajes de Programación 1

12. Importar el módulo `calculos.py` desde el sub-paquete `utilidades`, ejecutando:

```
from tostadas_pipo.utilidades import calculos
```

13. Importar la función `impuesto_iva14()` desde el módulo `impuestos.py` en el sub-paquete `utilidades`, ejecutando:

```
from tostadas_pipo.utilidades.impuestos import impuesto_iva14
```

14. Por ejemplo, cree un módulo llamado `calculo_factura_pipo.py`, que contiene las importaciones del paquete `tostadas_pipo`:

15. Archivo `calculo_factura_pipo.py` incluye el siguiente código:

```
from tostadas_pipo.utilidades import calculos
from tostadas_pipo.utilidades.impuestos import impuesto_iva14

monto = int(input("Introduzca un monto entero: "))
monto_suma = int(input("Introduzca un monto entero a sumar: "))

suma = impuesto_iva14(monto) + calculos.suma_total(monto_suma)

print "Total a Facturar: {0} BsS, con IVA 14%.".format(suma)
```

V

EJERCICIOS PROPUESTOS

Actividad Grupal

1. Desarrolle una aplicación en lenguaje Python que nos permita manejar un carrito de compra, la aplicación debe tener un menú de opciones y manejarse mediante módulos y paquetes.
2. Desarrollen una aplicación en Python con temática libre (por ejemplo: sistema de gestión de nota, juego educativo, app de productividad, herramienta científica, etc.). La aplicación debe estar correctamente estructurada utilizando módulos y paquetes, y debe incluir lo siguiente:
 - Descripción general del propósito de la aplicación y del problema que resuelve.
 - Identificación de los módulos y paquetes necesarios para organizar las funcionalidades del sistema.
 - Definición de las funciones principales de cada módulo, indicando:
 - Nombre de la función.
 - Descripción clara de su propósito.
 - Parámetros de entrada y su tipo.
 - Valor de retorno.
 - Flujo general de la aplicación: cómo se conectan los módulos entre sí.
 - Breve explicación de la estructura de carpetas y archivos del proyecto.

- Uso de estructuras de datos apropiadas según el tipo de aplicación (listas, diccionarios, clases, archivos, etc.).
- Agregar un menú para la interacción.

V**CUESTIONARIO**

1. ¿Qué es un módulo?
2. ¿Qué es el diseño modular?
3. ¿Qué ventajas ofrece el uso de módulos?
4. ¿Para qué son útiles los módulos?
5. ¿Cómo hacemos uso de un módulo?
6. ¿De qué forma se importa un módulo?
7. ¿Cómo se hace la importación de un módulo con import?
8. ¿Cómo se hace la importación con from?
9. ¿Cuándo hacemos uso de import?
10. ¿Cuándo hacemos uso de from para la importación?
11. ¿En qué consiste la descomposición funcional?
12. ¿En qué consiste el diseño modular?
13. ¿Qué es un path de búsqueda?
14. ¿Cómo se escribe un path de búsqueda?
15. ¿Qué es el bytecode?
16. ¿Cómo se genera el bytecode?
17. ¿Cómo se usa el bytecode?
18. ¿Cómo se ejecuta el módulo?
19. ¿Cómo se carga un módulo?
20. ¿Qué es un espacio de nombres?
21. ¿En qué consiste la importación?
22. ¿Qué función cumple un archivo `__init__.py`?
23. ¿Cómo se configura la variable `__all__`?
24. ¿Qué es un submódulo?
25. ¿Cómo se hace uso de `*`?
26. ¿Cómo se hacen las Referencias internas en paquetes?
27. ¿Cómo se mantienen Paquetes en múltiples directorios?
28. ¿Qué es un import relativo explícitos?
29. ¿Qué es un import relativo implícitos?

VI**BIBLIOGRAFIA Y REFERENCIAS**

- Fernández Montoro, Arturo, "Python 3 al Descubierto", 2da Edición, Editorial ALFAOMEGA, México, Año 2013.
- Horstmann, Cay S., Nectaise, Rance D. "Python for Everyone", 2nd Edition, John Wiley & Sons, Inc., New Jersey, Año 2014.
- Ascher, David, Lutz, Mark, "Learning Python", 2da Edición, O'Reilly Media, Inc., Sebastopol, CA, Año 2003.
- Barry, Paul, Griffiths, David, "Head First Programming", O'Reilly Media, Inc., Sebastopol, CA, Año 2009